CSC 255

Unit 9 Programming Assignment 1 [50 points]

RSA Encryption/Decryption

Encrypt and decrypt messages using a $key(n, e)$, where $n = pq$, and p and q are distinct prime numbers with four (4) digits each, and e is relatively prime to $(p-1)(q-1)$. Two numbers are relatively prime if their greatest common divisor is 1.

Use the **Euclidean Algorithm** to verify: $\gcd(e, (p-1)(q-1)) = 1$.

```
function gcd(e, m)
    x = e, y = m
    while y ≠ 0
        r = x mod y
        x = y
        y = r
    return x
```

Write a program that encrypts the following message: **[25 points]**

"The camera is hidden in the bushes"

Covert all letters to uppercase, remove all whitespace, and build blocks of four (4) letters starting at the beginning of the message until the end.

For example: **"THEC"**, would be the first block to encrypt. Notice, we will concatenate the four (4) letters after removing spaces. If there are empty spaces at the end of a block, pad the end with the letter "X". Given the above message, the final block would be: **"SHES"**, so no padding would be required.

When using blocks of four (4) letters (8 digits each block), because each letter is encoded using ASCII encoding, 65 to 90 (65 – A, 67 – B, ..., 90 – Z), our max encoded block would be 90909090. Any block, modulo n, is an 8-digit number since we will choose n such that it is greater than 90909090 and less than 9090909090. Therefore, the encrypted blocks will remain within our domain of four (4) letters.

Encryption

M = the encoded (plaintext) block of four (4) letters

C = the encrypted (cyphertext) block of four (4) letters

$C = M^e mod\ n$

Use the following algorithm for **modular exponentiation** to compute M^e:

```
function modular_pow(base, exponent, modulus)
    if modulus == 1 then return 0
    c = 1
```

```
    for e_prime = 0 to exponent-1
        c = (c * base) mod modulus
    return c
```

Decryption

$d =$ inverse of e modulo $(p-1)(q-1)$, which means $d \cdot e \equiv 1\ (mod(p-1)(q-1))$

M = the decrypted block of four (4) letters

C = the encrypted block of four (4) letters

$M = C^d mod\ n$

If we solve the following identity: $ed + mt = 1$ for d and t, where $m = (p-1)(q-1)$, d will be the inverse of e.

Use the following **extended Euclidean algorithm** to compute d which is a positive integer (d will assume the value of `old_s`, which is the return value of this function):

```
function extended_gcd(e, m)
    s = 0, old_s = 1, t = 1, old_t = 0, r = m, old_r = e
    while r ≠ 0
        quotient = old_r / r
        temp = r
        r = old_r – quotient * r
        old_r = temp
        temp = s
        s = old_s – quotient * s
        old_s = temp
        temp = t
        t = old_t – quotient * t
        old_t = temp
    if old_s < 0
         old_s = old_s + m
    return old_s
```

Note: The computation for `quotient` should be the truncated result of `old_r / r`.

Review the following example which shows how to solve for the inverse. This will help you understand how the algorithm above works.

Find a pair of integers (d, t) such that $13d + 2436t = 1$.

First, we perform the Euclidean algorithm to get:

$2436 = 187 \cdot 13 + 5$

$13 = 2 \cdot 5 + 3$

$5 = 1 \cdot 3 + 2$

$3 = 1 \cdot 2 + 1$

$2 = 2 \cdot 1 + 0$

This tells us the GCD of 13 and 2436 is 1.

Now work backwards from the equation with a remainder of 1 (i.e., the GCD) and use substitutions:

$1 = 3 - 1 \cdot 2$

$1 = 3 - 1 \cdot (5 - 1 \cdot 3)$

$1 = 2 \cdot 3 - 1 \cdot 5$

$1 = 2 \cdot (13 - 2 \cdot 5) - 1 \cdot 5$

$1 = 2 \cdot 13 - 4 \cdot 5 - 1 \cdot 5$

$1 = 2 \cdot 13 - 5 \cdot 5$

$1 = 2 \cdot 13 - 5 \cdot (2436 - 187 \cdot 13)$

$1 = 2 \cdot 13 - 5 \cdot 2436 + 935 \cdot 13$

$1 = 937 \cdot 13 - 5 \cdot 2436$

So, we now know the inverse of 13 is 937 (i.e., $d = 937$).

Now write a program that decrypts the message in blocks of four (4) letters**. [25 points]**

Because this cryptosystem scheme uses blocks of letters, it is referred to as a block cipher.

Your program will write the encrypted message to a file (*encrypt.rsa*). Finally, read the file (*encrypt.rsa*) with the encrypted message, decrypt it, and then write the results to a new file (*decrypt.rsa*).

Note: The block size can vary for this technique. If we change the block size to some n, we should also change the number of digits in p and q to n. This will ensure the encrypted block will be of size n.

Warning: Overflow can occur in the encryption step, so save time by confirming a block of four (4) letters is encrypted correctly before attempting to encrypt the entire message. Also, due to complications with overflow, always confirm the value of d is correct before proceeding to decrypt.